# 6. Release 3 - Requirements

# Project 1 - Release 3 Requirements

## Shared Projects, Contributor Evidence, and Basic Playable Demos

## Release Question

> **Can collaborative work become credible shared evidence that visitors can actually experience?**

Students can now maintain profiles. Release 3 makes projects editable, collaborative, and useful as evidence.

Release 1 and Release 2 requirements remain cumulative unless explicitly changed.

## Core Journeys

### Student

**My Projects -> Add Project -> Import from GitHub or create manually -> Invite teammate -> Publish**

### Contributor

**Receive invitation -> Review project/role -> Accept or decline -> Maintain own contribution statement**

### Public Visitor

**Student profile -> Shared project -> Understand contributors/roles -> Play demo / Watch demo / View source**

## Provided Fixtures

?

Use: standard_release.json, project_duplicate_access.json, working_demo.html

← Previous Next →

all in **Release 3 Fixtures**, in the Release 3 section. Choose **Download folder** and extract the ZIP; Moodle supplies these files together without the repository `demo` subfolder. Host `working_demo.html` on your own deployment or static host and use its public URL as the playable link. Upload/host `fallback.svg` as the fallback image. A local file path or Moodle login-protected download is not a public demo URL.

The working demo is deliberately small. Release 3 is about the project/deployment relationship, not building an XR application inside the portfolio project.

# Requirements

## Project Creation

**R3.01 - Add Project** - A verified student can create a project using one clear `Add Project` workflow.

**R3.02 - Manual Creation** - The workflow supports manual entry of at least title, short description, domain/type, project technologies, date/status, public links, and media/demo information.

**R3.03 - GitHub Import** - The student can provide/select a public GitHub repository and use repository metadata to propose useful project fields such as name/title, description, repository URL, and language/technology evidence. Imported values remain editable suggestions.

**R3.04 - Repository Is Not Project Identity** - A project remains valid if a repository is disconnected or changed. A project can contain zero, one, or multiple repository links.

## Shared Project and Maintainer

**R3.05 - Canonical Shared Project** - A collaborative project is one shared record referenced by multiple student profiles, not separately duplicated project copies for each contributor.

**R3.06 - Initial Maintainer** - The creator becomes the initial project maintainer and can edit shared project information.

Who else may edit shared project fields is your team's decision, but it must be a decision: state it in your release notes. "Authorized editor" below means whoever your model grants that right to. Accepted contributors always control their own contribution statement (R3.11) whether or not they can edit shared fields.

**R3.07 - Transfer Maintenance** - The maintainer can transfer maintenance to another

← Previous Next →

**R3.08 - Invite Contributor** - A maintainer can invite an existing platform student and propose a standardized role.

**R3.09 - Accept / Decline** - The invited student can accept or decline. They must not be publicly represented as an accepted contributor before acceptance.

**R3.10 - Contributor Role** - An accepted contributor has a standardized role plus an optional personal role/contribution description.

**R3.11 - Personal Contribution Control** - A contributor controls their own personal contribution statement. No other contributor or maintainer can silently rewrite it.

**R3.12 - Feature/Hide Personally** - A contributor can choose whether the project is featured in their portfolio presentation without deleting the underlying accepted relationship.

**R3.13 - Removal Protection** - A maintainer cannot silently erase an accepted contributor and their historical relationship. A contributor may remove themselves from personal display.

A disputed removal of the canonical relationship must be **recoverable**: the relationship is not destroyed by a single unilateral action, and it can be restored. A soft-delete a maintainer can reverse satisfies this; so does a two-party confirmation. Choose one and say which in your release notes.

## Skills and Evidence

**R3.14 - Project Technologies Are Separate** - Project-level technologies describe the project and do not automatically become personal skills for every contributor.

**R3.15 - Contributor Evidence** - Public project presentation makes each accepted contributor's role understandable.

## Duplicate and Concurrent Editing

**R3.16 - Duplicate Warning** - When creating a project with a repository URL already connected to an existing project, the user is warned and can view/join the existing project instead of accidentally creating a duplicate.

Name similarity may be used as an additional warning but must not automatically merge records.

**R3.17 - No Silent Lost Update** - If two authorized users edit the same shared project from stale versions, one save must not silently overwrite the other's newer change. A conflict warning/reload/merge approach is acceptable.

← Previous Next →

**R3.18 - Add Demo** - A project can store presentation links separately for at least: - playable web URL; - video/demo URL; - source/repository URL.

Not every project must have every type.

**R3.19 - No Arbitrary Embed Code** - Students provide URLs/structured information, not arbitrary raw iframe/script HTML that is injected into public pages.

**R3.20 - Visitor Action** - Where a playable URL exists, the public page offers a clear action such as `Play Demo` or `Launch Experience`.

**R3.21 - Fallback Evidence** - A project can provide a fallback image/video/link so that it remains understandable when the playable experience is unavailable or unsupported.

**R3.22 - Broken Demo Isolation** - A missing/broken demo URL does not destroy the project page, student profile, or site navigation. The visitor can continue using the platform and reach fallback evidence.

**R3.23 - Mobile Project Page** - At 390 CSS pixels, project details, contributor roles, and fallback evidence remain usable even if the interactive demo itself is unsuitable for mobile.

## Authorization / Regression / Release

**R3.24 - Project Authorization** - An authenticated student who is neither the maintainer nor an authorized editor cannot update another project's shared fields by changing a project identifier in a request.

**R3.25 - Profile and Discovery Regression** - R1/R2 directory, privacy, profile publication, customization, and employer inquiry continue working.

**R3.26 - Stable Project URL** - The canonical project retains one stable public identity/URL when maintenance is transferred or repositories change.

**R3.27 - Assessed Version** - The release is tagged `R3-submission` or instructor-approved equivalent.

# Explicit Non-Requirements for Release 3

You are not yet required to build: - staff verification of official projects; - company/partner administration; - full audit-history UI; - sophisticated duplicate merging; - universal XR/device support; - advanced capability detection or heavy-runtime performance optimization.

Those are not reasons to make the basic project/demo workflow unreliable.

← Previous Next →

A student should understand `Add Project`, `Add Teammate`, `Accept`, and `Play Demo` without learning your internal permission/status model.

# Required Release Evidence

Submit: - deployed URL; - repository URL; - `R3-submission` tag; - release notes/known issues; - one shared published project with at least two accepted contributors for QA; - one project containing a working demo/fallback; - one public repository used to demonstrate GitHub import; - `release_submission.md`, from the template in **Submission Templates** (Team Project Bootcamp section), with `R3-submission` as the tag; - completed `evaluation_adapter.json`. Start from `evaluation_adapter_template.json` in the same place and extend it with the routes this release adds — project creation and claiming, contributor roles, the duplicate warning path, and the demo/fallback surface. The adapter tells peer, agent, instructor, and any automated evaluator where your implementation exposes behaviour these requirements already demand. It is route/setup evidence, not proof that the behaviour passed, and it does not prescribe your framework, schema, or URL design.

← Previous Next →